

Real-Time Traffic Light Detection on Adaptive AUTOSAR Edge Devices Using a MobileNetV4-Backbone YOLO Model and Google LiteRT

Huy Tai Nguyen*, Dang Duy Nguyen*, Si Hoang Le, Hoang Phuc Do,
Dang Huy Le, Van Sy Phan, Trong Nhan Le†

*Faculty of Computer Science and Engineering
Ho Chi Minh City University of Technology (HCMUT) – VNU-HCM
Ho Chi Minh City, Vietnam*

Email: {tai.nguyenraphael, duy.nguyendang04, hoang.lesi184, phuc.dohoang001,
huy.ledang265, syphan.cse, trongnhanle}@hcmut.edu.vn

Abstract—This paper presents a lightweight edge AI framework for real-time traffic light detection integrated into the AUTOSAR Adaptive Platform for Advanced Driver Assistance Systems (ADAS). To meet the strict latency and resource constraints of execution on AI edge sensor nodes, we propose a YOLO11-MobileNetV4 architecture optimized via post-training quantization for efficient CPU-only inference. Implemented as a Service-Oriented Architecture (SOA) compliant C++ Adaptive Application, the perception pipeline integrates seamlessly into the AUTOSAR environment. Experimental results show that our model achieves a real-time throughput of approximately 27 frames per second on embedded hardware. It delivers a $2.5\times$ speedup over the YOLO11n baseline and surpasses both YOLO8n and SSD-MobileNetV3 in inference speed, while maintaining a higher mAP@50 than the SSD baseline. This validates the feasibility of deploying scalable, accelerator-free edge perception services on general-purpose hardware for cost-effective ADAS development.

Index Terms—AUTOSAR Adaptive Platform, Edge AI for Sensing, Object Detection, YOLO-MobileNetV4, Service-Oriented Architecture, Embedded Deep Learning.

I. INTRODUCTION

Traffic Light Detection (TLD) is a key perception task in Advanced Driver Assistance Systems (ADAS), where reliable real-time interpretation of traffic signals is essential for safe vehicle operation [1]. To meet stringent latency and bandwidth requirements, modern automotive systems increasingly push perception intelligence closer to sensors, adopting edge computing paradigms. In this context, the AUTomotive Open System ARchitecture (AUTOSAR) Adaptive Platform has emerged as a standardized framework for deploying service-oriented perception applications on automotive edge devices [2].

Deploying deep learning-based perception on such resource-constrained edge nodes remains challenging. State-

of-the-art object detectors from the YOLO family [3][4] are typically optimized for GPUs or NPUs, which conflicts with the cost, power, and thermal constraints of sensor-level devices. Recent works address this issue using lightweight backbones and optimized runtimes. In particular, YOLO models with MobileNetV4 backbones achieve favorable accuracy-efficiency trade-offs for CPU-based inference [5][6], while lightweight runtimes such as Google LiteRT enable efficient on-device execution through model conversion and quantization [7]. However, most existing studies focus on standalone inference and overlook integration within standardized automotive middleware.

This paper proposes a lightweight and standardized traffic light detection framework for automotive edge devices based on the AUTOSAR Adaptive Platform. A quantized YOLO-MobileNetV4 model is deployed using the Google LiteRT C++ runtime and exposed as a service-oriented perception component via SOME/IP communication [8]. The implementation leverages the PopcornSAR toolchain to ensure AUTOSAR compliance without relying on specialized hardware accelerators.

The main contributions of this work are:

- A YOLO-MobileNetV4 detector optimized for CPU-only inference on sensor-class edge devices.
- A service-oriented integration of LiteRT-based deep learning inference within the AUTOSAR Adaptive Platform.
- An experimental validation demonstrating cost-effective, accelerator-free edge perception for ADAS.

To position the proposed work within existing research, the following section reviews related studies in three complementary areas: the evolution of the AUTOSAR Adaptive Platform, lightweight YOLO-based object detection for edge devices, and traffic light detection with an emphasis on sustainable edge inference. This review highlights the limitations of current

*H. T. Nguyen and D. D. Nguyen contributed equally to this work.

†Corresponding author: Trong Nhan Le (trongnhanle@hcmut.edu.vn)

approaches and motivates the need for a unified, standardized edge AI solution.

II. RELATED WORK

Existing studies span three closely related domains: the AUTOSAR Adaptive Platform and its tooling ecosystem, lightweight object detection for automotive edge devices, and edge AI-enabled traffic light detection at the sensor level with a focus on inference sustainability. Despite notable advances in each area, their convergence into a unified and AUTOSAR-compliant edge perception framework remains underexplored.

A. AUTOSAR Adaptive Platform and Tooling Ecosystem

The AUTOSAR Adaptive Platform was introduced to support service-oriented and high-performance automotive applications within Software-Defined Vehicle (SDV) architectures [2][9]. By providing hardware abstraction, dynamic deployment, and scalable communication mechanisms, it enables the implementation of computation-intensive ADAS functions. Toolchains such as PopcornSAR further facilitate practical development by supporting authoring, configuration, and code generation of AUTOSAR-compliant Adaptive Applications.

Although AUTOSAR Adaptive Platform provides a standardized middleware foundation, it does not inherently mitigate the computational burden of deep learning inference. Consequently, perception models deployed on the platform must be sufficiently lightweight to operate on general-purpose CPUs, motivating the exploration of efficient detection architectures and optimized inference pipelines.

B. Lightweight Object Detection for Automotive Applications

The YOLO family is widely adopted for real-time, resource-constrained object detection due to its favorable accuracy-latency trade-offs [3][4][10][11]. Prior iterations, notably YOLOv8, have established strong baselines for edge tasks by maintaining high precision alongside acceptable inference latency, often optimized through custom dataset fine-tuning and parameter pruning [12][13]. Alternatively, the Single Shot MultiBox Detector (SSD) coupled with the MobileNetV3 backbone remains a standard benchmark for mobile vision. Highly favored for its minimal computing power requirements and efficient feature extraction via depthwise separable convolutions [14][15], this lightweight combination has proven highly effective for practical deployment on low-power single-board computers, such as the Raspberry Pi, serving as a reliable baseline for resource-constrained environments [15].

While these foundational models provide robust performance, recent studies emphasize further architectural optimizations to meet the strict latency and safety requirements of automotive edge devices. Replacing standard YOLO backbones with the advanced MobileNetV4 architecture has been shown to significantly improve inference efficiency for embedded deployment [5][6][16]. However, most existing works focus on standalone model optimization or hardware-specific compression without addressing standardized automotive middleware integration. Furthermore, they rarely provide direct comparisons against established baselines such as YOLOv8,

YOLO11, and SSD-MobileNetV3 in a unified embedded environment.

C. Traffic Light Detection and Edge Inference Sustainability

Traffic light detection poses additional challenges due to small object size and varying illumination conditions. While algorithmic enhancements can improve detection accuracy, they often increase model complexity, limiting suitability for cost-sensitive edge devices. Beyond algorithm design, recent studies emphasize inference sustainability by analyzing the trade-offs among accuracy, latency, and energy consumption on constrained platforms [17].

Lightweight inference runtimes such as Google LiteRT (TensorFlow Lite) have therefore gained attention for enabling efficient on-device execution through model conversion and quantization [7][18][19][20]. Despite advances in lightweight models, optimized runtimes, and AUTOSAR-based deployment frameworks, these aspects are typically studied in isolation.

In contrast, this work targets their intersection by jointly synthesizing the structural efficiency of the MobileNetV4 backbone, the runtime sustainability enabled by LiteRT-based quantization, and the standardized deployment capabilities of the AUTOSAR Adaptive Platform. The following section presents the proposed methodology, detailing the hybrid YOLO-MobileNetV4 architecture and the optimized edge-centric inference pipeline.

III. METHODOLOGY: HYBRID YOLO-MOBILENET ARCHITECTURE FOR EDGE INFERENCE

Deploying deep learning-based object detection on resource-constrained edge devices requires balancing detection accuracy and computational efficiency. To address this challenge, the proposed methodology combines a YOLO detection head with a lightweight MobileNetV4 backbone and an optimized on-device inference pipeline, targeting efficient CPU-based deployment on embedded platforms.

A. YOLO-MobileNetV4

This subsection presents the hybrid YOLO-MobileNetV4 design for CPU-centric edge inference, including the MobileNetV4 backbone, the motivation for its integration with YOLO, and a concise backbone-replacement procedure within the Ultralytics framework.

1) *MobileNetV4*: MobileNetV4 demonstrates strong suitability for CPU-based edge devices due to its favorable accuracy-latency trade-offs on general-purpose processors. As shown in Fig. 1, MobileNetV4 consistently achieves near-Pareto-optimal performance on mobile CPUs (e.g., Galaxy S23), outperforming earlier MobileNet variants at comparable latency. Its Universal Inverted Bottleneck (UIB) design enables efficient integer arithmetic and regular memory access, making MobileNetV4-Small well suited for real-time inference under strict latency and power constraints [6]. These efficiency characteristics motivate the integration of MobileNetV4 into the YOLO framework, one of the most widely adopted architectures for practical sensor-class deployment.

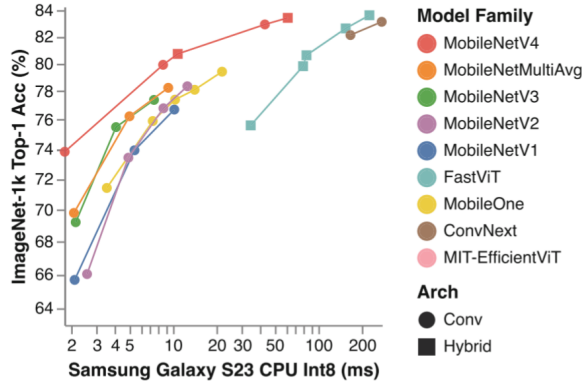


Fig. 1. MobileNetV4 models exhibit near-Pareto-optimal accuracy–latency trade-offs on Samsung S23 device [6].

2) *YOLO–MobileNetV4 Integration for Sensor-Class Devices*: The YOLO family is widely adopted for real-time object detection due to its efficient heads and multi-scale design [5][6][16]. However, conventional YOLO backbones based on CSP/C3 blocks are computationally intensive and unsuitable for CPU-only sensor-class devices. To enable efficient embedded inference, the original YOLO11 backbone is replaced with MobileNetV4, preserving multi-scale detection while significantly reducing computational cost.

The integration follows the modular design of the Ultralytics YOLO framework, where backbone, neck, and detection head are decoupled. In summary, MobileNetV4 is implemented as a custom backbone module (`ultralytics/nn/mobilenetv4.py`) exposing P3–P5 feature maps, registered in the model factory (`ultralytics/nn/tasks.py`), and aligned with the YOLO neck using lightweight 1×1 adapters when required. A custom YAML configuration (`yolo11_mnv4.yaml`) replaces the original C3k2/Conv backbone with MobileNetV4-ConvSmall, and the integrated model is validated prior to ONNX and TFLite export.

B. Implementation: YOLO–MobileNetV4 on LiteRT

To enable efficient CPU-only inference on sensor-class edge devices, this work adopts LiteRT (formerly TensorFlow Lite) and implements the complete inference pipeline in C++ to minimize runtime overhead and reduce CPU inference latency. Unlike common Python-based deployments, the proposed C++ implementation leverages LiteRT’s static tensor allocation and FlatBuffers-based model serialization to achieve deterministic and low-memory execution. In accordance with edge AI sustainability principles [17], the YOLO11–MobileNetV4 model is converted to the `.tflite` format and optimized via post-training quantization, enabling efficient deployment without dedicated accelerators. To the best of our knowledge, this work represents one of the first end-to-end implementations that combines a YOLO11–MobileNetV4 detector with a fully C++ LiteRT runtime for real-time inference on CPU-based

sensor devices. The overall inference workflow is summarized in Algorithm 1.

Algorithm 1 LiteRT-based YOLO11–MobileNetV4 Inference Pipeline

Require: Input frame I , LiteRT model M

Ensure: Final detections $\mathcal{D}$

- 1: // — Initialization —
- 2: Load `.tflite` model M
- 3: Create LiteRT interpreter and allocate static tensors
- 4: Configure CPU threads for inference
- 5: // — Preprocessing —
- 6: Compute letterbox scale and padding
- 7: Resize I and pad to network input size
- 8: Convert color space and normalize pixel values
- 9: Copy preprocessed data to input tensor
- 10: // — Inference —
- 11: Invoke LiteRT interpreter forward pass
- 12: // — Postprocessing —
- 13: **for** each prediction p in output tensor **do**
- 14: Decode bounding box and confidence score
- 15: Discard p if confidence $< \tau_c$
- 16: Map box coordinates back to original image space
- 17: Append valid detection to proposal set
- 18: **end for**
- 19: Apply class-wise Non-Maximum Suppression with threshold τ_{iou}
- 20: // — Output —
- 21: Encode final detections and publish to sensor interface
- 22: **return** $\mathcal{D}$

Although the proposed YOLO–MobileNetV4 architecture enables efficient edge inference, its practical deployment in automotive systems requires a standardized and safety-aware software framework. The following section therefore introduces the AUTOSAR Adaptive Platform and discusses its role in supporting efficient and reliable execution of edge AI perception services.

IV. AUTOSAR ADAPTIVE PLATFORM

The AUTOSAR Adaptive Platform serves as standardized middleware for sensor-level edge AI perception on automotive edge devices, supporting efficient execution and safety-aware integration.

A. AUTOSAR Overview

AUTOSAR is a standardized automotive software framework that improves scalability and portability of Electronic Control Unit (ECU) software by decoupling application logic from hardware-specific implementations. While the AUTOSAR Classic Platform is well suited for deeply embedded, safety-critical control systems, its static configuration model and limited support for dynamic workloads and high-bandwidth data processing restrict its applicability to modern perception-driven applications.

B. AUTOSAR Adaptive Platform for Edge Computing Devices

To support computationally intensive applications such as ADAS and on-device perception, AUTOSAR introduced the *Adaptive Platform (AP)*, which targets automotive edge computing devices operating close to sensors. AUTOSAR Adaptive Platform runs on POSIX-compliant operating systems (e.g., Linux or QNX), enabling direct integration of modern C++ software stacks and deep learning inference runtimes such as Google LiteRT.

From a performance standpoint, AUTOSAR Adaptive Platform supports multi-core execution and resource-managed runtime environments, allowing AI perception tasks to run as independent Adaptive Applications with predictable latency. By executing perception locally at the sensor edge, the platform reduces communication overhead and improves real-time responsiveness. In terms of safety, process isolation, lifecycle management, and health monitoring enable fault containment and improve system robustness, which are essential for deploying AI-based perception in automotive environments.

Key features relevant to this work include [9]:

- *Service-Oriented Architecture (SOA)*: Perception results are exposed as reusable services using standardized protocols such as SOME/IP [8].
- *Edge Computing Support*: Resource-managed execution on multi-core edge ECUs enables low-latency AI inference.
- *Modern C++ Environment*: Standardized C++ APIs support the implementation and deployment of computer vision and deep learning pipelines.

In this work, AUTOSAR Adaptive Platform serves as standardized middleware that enables real-time, efficient, and safety-aware deployment of deep learning-based traffic light detection directly on automotive edge devices. Building upon the AUTOSAR Adaptive Platform concepts discussed above, the following section presents the concrete system implementation, detailing how the proposed edge AI perception pipeline is realized as Adaptive Applications, integrated via SOME/IP communication, and executed in real time on an automotive edge platform.

V. REAL-TIME EVALUATION AND MODEL BENCHMARKING

This section details the deployment of the quantized YOLO-MobileNetV4 model onto an AUTOSAR Adaptive Platform. The integration process involves establishing a toolchain for service-oriented architecture configuration, designing the system topology, and evaluating the inference performance on the target edge device unit.

A. Toolchains and Workflow Configuration

Leveraging the PopcornSAR ecosystem introduced in Section II, the system was implemented following a model-based development workflow. System modeling and service configuration were performed using the *AutoSAR.io* authoring tool, while application development and runtime support relied on the *PARA SDK*.

At a high level, the workflow consists of three main steps. First, the Adaptive Application architecture and service interfaces were defined in *AutoSAR.io*, including provider-consumer relationships for camera input and perception output, as well as event-based communication over SOME/IP [8]. Second, the configured *.arxml* descriptions were used to automatically generate C++ service skeletons and proxies, ensuring compliance with AUTOSAR communication standards. Finally, the generated artifacts were integrated with the *PARA SDK* runtime libraries, where the LiteRT inference engine was embedded into the application lifecycle to enable real-time perception within a service-oriented execution environment. Based on the configured toolchain and model-based workflow, our proposed approach will be presented.

B. Proposed Approach

The proposed approach, illustrated in Figure 2, follows a Producer-Consumer communication pattern implemented via SOME/IP [8] and is designed around an edge AI-centric sensing paradigm.

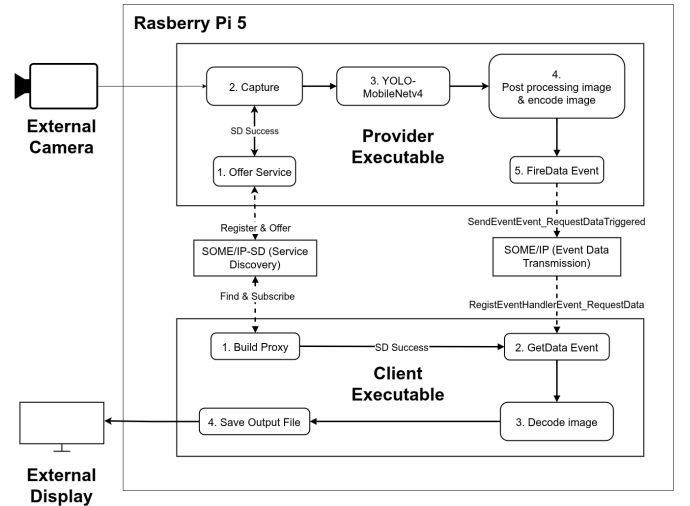


Fig. 2. Proposed approach of the AUTOSAR Adaptive Platform Traffic Light Detection System on Raspberry Pi 5.

The system consists of two Adaptive AUTOSAR Applications deployed on the edge device, where intelligence is pushed directly to the sensor node to minimize data transmission and latency:

- *Provider Executable (Send)*: Interfacing directly with an external camera, this edge node executes a five-stage pipeline: (1) offering the service via SOME/IP-SD; (2) capturing live video frames; (3) performing real-time YOLO-MobileNetV4 inference; (4) post-processing the annotated image and encoding it into a vector; and (5) triggering a data event (*SendEvent_RequestDataTriggered*) to transmit the vector data.
- *Client Executable (Receive)*: Acting as the service consumer, it reconstructs the transmitted data by:

(1) finding the service and building a proxy via SOME/IP-SD; (2) handling the received data event (`RegistEventHandler_RequestData`); (3) decoding the vector into a image; and (4) saving the output for external display rendering.

- *SOME/IP Communication*: This middleware manages dynamic service registration and discovery via SOME/IP-SD. To handle the transmission of heavy encoded image payloads, the SOME/IP Transport Protocol (SOME/IP-TP) is utilized to segment and reassemble large messages across the network.

To validate the effectiveness of the proposed architecture, the following section evaluates the system’s real-time performance and detection quality on the embedded edge device.

C. Dataset and Training Setup

To evaluate our object detection models, we created a dataset named *Traffic Lights*, which contains 6,536 images and 9,738 annotated traffic light instances under diverse, real-world conditions. The dataset was partitioned into training (5,712 images, 87.4%), validation (548 images, 8.4%), and testing (276 images, 4.2%) sets, with no empty frames. Annotations are categorized into three classes exhibiting a natural imbalance: Green (50.3%), Red (35.0%), and a minority Yellow class (14.7%).

During preprocessing, all images were resized to 320×320 pixels and normalized to the $[0, 1]$ range. To mitigate the class imbalance and enhance model generalization, we applied Mosaic data augmentation during training. This technique combines four distinct images into a single frame to simulate varying object scales and partial occlusions.

All experiments were conducted on a Windows 11 workstation. The software environment was isolated using a Python virtual environment (`venv`), and model training was accelerated using an NVIDIA GeForce RTX 4060 8GB GPU.

D. Compared Models

To analyze the speed-accuracy trade-off for embedded traffic light detection, four models were evaluated: YOLO11n, YOLO11-MobileNetV4 (ConvSmall), YOLO8n, and SSD-MobileNetV3.

1) *Detection Accuracy*: As detailed in Table I, the standard YOLO architectures (YOLO11n, YOLO8n) achieve the highest detection accuracies, all exceeding 0.97 in mAP@50. Replacing the backbone with MobileNetV4 in YOLO11 results in an expected accuracy trade-off, reducing the mAP@50 from 0.977 to 0.929 (a relative decrease of approximately 4.9%). However, YOLO11-MobileNetV4 still outperforms the traditional SSD-MobileNetV3 baseline (mAP@50 of 0.743), demonstrating sufficient detection performance for the targeted traffic light recognition task.

2) *Inference Speed*: Inference latency is the primary bottleneck for edge deployment. According to the experimental results summarized in Table I, the original YOLO11n in LiteRT format requires approximately 90 ms per frame. While older architectures like YOLO8n improve this to roughly

78 ms, the YOLO11-MobileNetV4 model significantly reduces inference latency to approximately 38 ms.

This represents a $\sim 2.4\times$ speedup (an approximately 58% reduction in latency) compared to the baseline YOLO11n. Furthermore, it achieves a faster inference time than the SSD-MobileNetV3 model (~ 44 ms) while maintaining a notably higher mAP@50 (0.929 vs. 0.743). This latency reduction enables real-time performance of approximately 26 FPS on embedded hardware, validating the suitability of the MobileNetV4 backbone for real-time edge inference.

TABLE I
PERFORMANCE TRADE-OFF BETWEEN DIFFERENT MODELS

Model	Format	mAP@50 (%)	mAP@50-95 (%)	Inference Time
YOLO11n	LiteRT	97.7	78.5	~ 90 ms
YOLO11n-MobileNetV4 (Ours)	LiteRT	92.9	72.3	~ 38ms
YOLO8n	LiteRT	97.4	81.1	~ 78 ms
SSD-MobileNetV3	ONNX	74.3	54.8	~ 44 ms

The experimental results demonstrate the feasibility and efficiency of the proposed approach for real-time traffic light detection on edge devices. Based on these findings, the final section summarizes the key contributions and discusses the implications for future AUTOSAR-based edge perception systems.

E. Result and Evaluation

The system was evaluated in terms of inference accuracy and real-time performance using a C++ Adaptive AUTOSAR implementation deployed on the above Deployment Setup.



Fig. 3. Traffic light detection results on YOLO11-MobileNetV4 model using the proposed AUTOSAR Adaptive Platform C++ implementation on a Raspberry Pi 5.

As shown in Fig. 3, the proposed system successfully detects traffic lights with accurate bounding box localization and correct class prediction (e.g., *Red*). Key performance metrics obtained from runtime logs reveal that the system consistently reports stable detection confidence values, such as 0.63 for the rightmost traffic signal. These qualitative results confirm that the integration of the LiteRT inference runtime within the Adaptive AUTOSAR framework operates correctly on embedded hardware, indicating effective quantization and

model integration without observable degradation in detection quality.

These findings demonstrate that real-time traffic light detection can be successfully achieved within a standardized Adaptive AUTOSAR architecture using low-cost, CPU-based embedded hardware, entirely without reliance on dedicated AI accelerators. This highlights the broader feasibility of deploying scalable and cost-efficient perception services directly into automotive edge systems. While the qualitative and runtime results confirm the operational feasibility of the proposed system, a comprehensive evaluation requires further context. Therefore, a quantitative comparison with alternative model configurations is necessary to thoroughly analyze the speed-accuracy trade-off for optimal embedded deployment.

VI. CONCLUSION

In this paper, a lightweight and standardized framework for traffic light detection on resource-constrained edge devices using the AUTOSAR Adaptive Platform has been proposed. By integrating the YOLO11 architecture with a MobileNetV4 backbone and utilizing the LiteRT runtime, the system successfully addresses the computational bottlenecks inherent to general-purpose edge CPUs. This contribution directly supports the advancement of Edge AI for sensing and robotics by demonstrating the ability to deliver real-time perception without relying on specialized AI accelerators. The proposed solution achieves an inference latency of approximately 30–40 ms on a Raspberry Pi 5, representing a $\sim 2.4\times$ speedup compared to the baseline YOLO11n. However, this efficiency entails a necessary trade-off in detection precision. While the lightweight model retains approximately 95 % of the baseline accuracy for general detection (a relative reduction of merely 4.9% in mAP@50), a notable limitation is observed in stricter localization metrics, with mAP@50-95 decreasing from 78.5% to 72.3%. Despite this limitation in bounding box tightness, the system maintains a stable throughput of 24–33 FPS, thereby validating the feasibility of deploying efficient Machine Learning for Sensor Systems within the cost-effective and scalable SDV ecosystem. Future research directions include investigating advanced lightweight models, such as hybrid Vision Transformers, and utilizing the Yocto Project for OS-level CPU optimizations to further reduce inference latency on both current and high-end edge devices.

ACKNOWLEDGMENT

We acknowledge Ho Chi Minh City University of Technology (HCMUT), VNU-HCM for supporting this study. The authors thank Dr. Trong Nhan Le for his technical guidance and supervision. We are grateful to PopcornSAR Co., Ltd for providing essential resources, and to Mr. Chae Seung-yueb and Mr. Lee Jae-yoon for their professional mentorship and industrial insights. Finally, the authors acknowledge Google Gemini for its generative AI assistance in refining the technical writing and LaTeX formatting of this manuscript.

REFERENCES

- [1] Y. Wang, D. Liu, and E. Matson, "Accurate perception for autonomous driving: Application of kalman filter for sensor fusion," in *2020 IEEE Sensors Applications Symposium (SAS)*, 2020, pp. 1–6.
- [2] AUTOSAR, "AUTOSAR Adaptive Platform Specifications," <https://www.autosar.org>, 2021, release R21-11. [Online]. Available: <https://www.autosar.org/standards/adaptive-platform>
- [3] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.
- [4] M. L. Ali and Z. Zhang, "The yolo framework: a comprehensive review of evolution, applications, and benchmarks in object detection," *Computers*, vol. 13, p. 336, 12 2024.
- [5] Y. Lei, C. Shan, W. Ge, X. Wang, and J. Hu, "emnv4-yolo: A high-efficiency target detection framework for robotic driving vehicle instrument reading," *IEEE Access*, vol. 13, pp. 47 123–47 131, 2025.
- [6] D. Qin, C. Lechner, M. Delakis, M. Fornoni, S. Luo, F. Yang, W. Wang, C. Banbury, C. Ye, B. Akin, V. Aggarwal, T. Zhu, D. Moro, and A. Howard, "Mobilenetv4: Universal models for the mobile ecosystem," *Lecture Notes in Computer Science*, vol. 15098, pp. 78–96, 11 2024.
- [7] G. Mannes, E. Manor, and S. Greenberg, "Lstm hardware inference accelerator for litert," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 73, no. 2, pp. 1052–1064, 2026.
- [8] AUTOSAR, "AUTOSAR SOME/IP Protocol Specification," <https://www.autosar.org>, 2021, release R21-11, Foundation Standard. [Online]. Available: https://www.autosar.org/fileadmin/standards/R21-11/FO/AUTOSAR_PRS_SOMEIPProtocol.pdf
- [9] —, "AUTOSAR Adaptive Platform Design Specification," <https://www.autosar.org>, 2021, release R21-11, Adaptive Platform. [Online]. Available: https://www.autosar.org/fileadmin/standards/R21-11/AP/AUTOSAR_EXP_PlatformDesign.pdf
- [10] J. Wei, A. As'arry, K. Anas Md Rezali, M. Zuhri Mohamed Yusoff, H. Ma, and K. Zhang, "A review of yolo algorithm and its applications in autonomous driving object detection," *IEEE Access*, vol. 13, pp. 93 688–93 711, 2025.
- [11] E. Saqib, I. S. Leal, I. Shallari, A. Jantsch, S. Krug, and M. O'Nils, "Optimizing the iot performance: A case study on pruning a distributed cnn," in *2023 IEEE Sensors Applications Symposium (SAS)*, 2023, pp. 1–6.
- [12] L. Guo, H. Liu, Z. Pang, J. Luo, and J. Shen, "Optimizing yolo algorithm for efficient object detection in resource-constrained environments," in *2024 IEEE 4th International Conference on Electronic Technology, Communication and Information (ICETCI)*, 2024, pp. 1358–1363.
- [13] T. Ahmed, A. Maaz, D. Mahmood, Z. u. Abideen, U. Arshad, and R. H. Ali, "The yolov8 edge: Harnessing custom datasets for superior real-time detection," in *2023 18th International Conference on Emerging Technologies (ICET)*, 2023, pp. 38–43.
- [14] S. Liu, H. Shi, and Z. Guo, "Remote sensing image object detection based on improved ssd," in *2022 3rd International Conference on Computer Vision, Image and Deep Learning & International Conference on Computer Engineering and Applications (CVIDL & ICCIA)*, 2022, pp. 421–424.
- [15] M. A. Adnan, F. Ahammed, M. F. Rahman, N. Alam, I. A. Prince, and M. T. Ahamed, "Raspberry pi-powered iot device for blind navigation using mobilenetv3-ssd image processing," in *2023 International Conference on Information and Communication Technology for Sustainable Development (ICICT4SD)*, 2023, pp. 274–279.
- [16] G. Chen, Z. Xia, X. Ma, Y. Jiang, and Z. He, "Mobilenet-gdr: a lightweight algorithm for grape leaf disease identification based on improved mobilenetv4-small," *Frontiers in Plant Science*, vol. 16, 11 2025.
- [17] P. Singh, G. R. V. S. Ponmaniraj, A. Aggarwal, and S. Kumari, "Energy-efficient iot architectures for smart cities: Implementing adaptive edge computing for sustainable urban development," in *2025 International Conference on Electronics, AI and Computing (EAIC)*, 2025, pp. 1–6.
- [18] A. Trpcheska, F. Zevnik, and S. Bader, "Towards real-time vision-based sign language recognition on edge devices," in *2024 IEEE Sensors Applications Symposium (SAS)*, 2024, pp. 1–6.
- [19] P. Du, T. Polonelli, M. Magno, and Z. Cheng, "Towards lightweight deep neural network for smart agriculture on embedded systems," in *2022 IEEE Sensors Applications Symposium (SAS)*, 2022, pp. 1–6.
- [20] M. Giordano, N. Baumann, M. Crabolu, R. Fischer, G. Bellusci, and M. Magno, "Design and performance evaluation of an ultralow-power smart iot device with embedded tinymml for asset activity monitoring," *IEEE Transactions on Instrumentation and Measurement*, vol. 71, pp. 1–11, 2022.